THE FIELD GUIDE / VOLUME 01

From zero to shipping real software.

Claude Code, GitHub, Vercel, Supabase and Obsidian, explained from scratch. No coding background required.



Build with Claude

AI projects, websites and front-end designs

15 chapters | copy-paste prompts | 2 free skills included | prices checked Nov 2026

ABOUT THE AUTHOR

Hi, I'm glad you're here.



I'm a full-time student who builds software with Claude. I've built and delivered paid admin-automation software for a company, designed websites for other people, and earned my **HTML + CSS Responsive Web Design certificate**.

My daily stack is **Claude Code, Vercel, Supabase and Obsidian**, and this guide teaches the exact workflow I use, start to finish, so you can go from knowing nothing to shipping real, working software.

I started my account to share the AI projects, cool websites and front-end designs I build with Claude, and to show anyone how to do the same. Everything in here is practical: no theory for theory's sake, just what to click, what to type, and what to say to Claude.

How to use this guide

If you're brand new, read in order: Part 1 sets you up, Part 2 teaches each tool, Part 3 has you build and ship a real app. If you already code a little, jump to the chapter you need. Every chapter has copy-paste prompts you can use as-is. Prices and features in this guide were checked in November 2026. They change often, so confirm on each company's pricing page before you pay.

INSIDE

What you'll learn

PART 1: FOUNDATIONS

01 **What vibe coding actually is**
The mindset that makes everything else work

02 **Subscriptions and costs: what to buy**
Every tier for Claude, Vercel, Supabase, GitHub and Obsidian, with recommended stacks

03 **Set up your machine**
Accounts and installs in the right order, plus 10 terminal commands

04 **Claude Code: terminal vs desktop app**
What each is, how they differ, and which to start with

PART 2: THE TOOLS

05 **GitHub: your safety net**
Version control without the jargon

06 **Vercel: put it on the internet**
Deploy, preview, env vars, domains

07 **Supabase: add a database and logins**
Tables, auth, keys, and row level security

08 **Obsidian: your project brain**
Notes, specs, and a prompt library Claude can read

09 **Claude skills**
Where to find them, how to install them, and two free downloads

PART 3: BUILD AND SHIP

10 **The build loop and prompting that works**
The 5-part prompt and before/after examples

11 **Capstone: build and ship a real app**
A login-protected bookmark vault, step by step

12 **Fix anything: the debugging playbook**
What to do when it breaks

13 **15 cool things to build**
Prompt starters you can steal today

14 **Pro habits and the pre-launch checklist**
Security, CLAUDE.md, and shipping with confidence

15 **Cheat sheet and glossary**
Screenshot this section

PART 1: FOUNDATIONS

CHAPTER 01

What vibe coding actually is

You describe what you want in plain English. Claude writes the code. You run it, react to it, and steer. That's the whole idea.

The term "vibe coding" was coined by AI researcher Andrej Karpathy in early 2025 for building software by talking to an AI instead of typing every line yourself. The key shift: **your job moves from writing syntax to directing a build.** You're the creative director, and Claude is a very fast, very patient engineer.

Why this works now

- **You can start from zero.** You don't need to know React or Python to ship something real. You need a clear picture of what you want.
- **Feedback is instant.** Describe, run, look, adjust. A full cycle takes a minute, not a week.
- **Taste is the bottleneck.** When everyone can generate code, the people who stand out have good ideas and a good eye.

What it is not

It is not "press a button and a perfect app appears." Your first draft will be roughly 70% right. The skill is the next ten minutes: spotting what's off, describing it clearly, and iterating. That skill is learnable, and this guide is built around it.

> **The one rule that matters most**
>
> Never ask for the whole app at once. Ask for the smallest working version, run it, then add one thing at a time. Every habit in this guide is a variation of that sentence.

The stack you'll learn

Tool	Job	Think of it as
Claude / Claude Code	Writes and edits the code, runs commands, fixes errors	Your engineer
GitHub	Stores every version of your code	Save game + undo button
Vercel	Puts your site on the internet with a URL	Your hosting and launch pad
Supabase	Database, user logins, file storage	Your app's memory
Obsidian	Notes, specs and prompts in plain files	Your project brain

Subscriptions and costs: what to buy

You can start for $0 and scale as your projects grow. Here is every tier and exactly when to upgrade.

Claude plans (individuals)

Plan	Price	Claude Code?	Best for
Free	$0	No	Trying Claude, chatting, building small Artifacts
Pro	$20/mo ($17/mo billed annually)	Yes	**Start here.** Daily building, Claude Code, Projects, Claude in Chrome
Max	$100/mo	Yes	5x Pro usage. For when you hit limits regularly
Ultra	$200/mo	Yes	Heaviest use: roughly 10x Pro weekly usage, no 5-hour limit on verified accounts

Usage on paid plans resets on a rolling 5-hour window plus a weekly limit, and the same pool is shared by chat and Claude Code. If you chat all afternoon, you'll have less left for coding. Team plans (Standard seat $20 to $25/mo, Premium seat $100 to $125/mo, for 2 to 150 people) include Claude Code and suit groups, not solo builders.

My recommendation

Start on **Pro ($20/mo)** and build for a few weeks. Upgrade to **Max ($100/mo)** only when limits interrupt you most days. Skip Ultra unless you're running Claude Code for hours every single day. Claude Code is not included on the Free plan, so Pro is the real entry point for this guide.

Claude API vs subscription (don't mix these up)

The **subscription** (above) is a flat monthly fee for using Claude and Claude Code yourself. The **API** (platform.claude.com) is pay-per-use billing for when your *app* calls Claude, for example an app with an "AI summarize" button. They're billed separately. You only need the API once you're building AI features into something other people use.

The rest of the stack

Tool	Free tier	Paid tier	When to upgrade
GitHub	Free: unlimited public and private repos	Paid plans for teams and extras	Almost never, for solo builders
Vercel	Hobby: $0, **personal / non-commercial use only**	Pro: $20/mo (includes $20 usage credit, free custom domain)	When the site makes money or serves clients
Supabase	Free: 2 active projects, 500 MB database, **pauses after 1 week of inactivity**	Pro: from $25/mo (8 GB disk, no inactivity pausing)	When real users depend on it staying up
Obsidian	Free for personal use, and no commercial license is required (work users are encouraged to buy one)	Sync $4/mo (annual), Publish $8/mo (annual), Commercial $50/user/yr	Only if you want phone sync or to publish notes

Recommended stacks

Stack	What you pay	Who it's for
Explorer	$0: Claude Free + GitHub + Vercel Hobby + Supabase Free + Obsidian	Learning with Artifacts and tiny projects. No Claude Code yet.
Builder (recommended)	**$20/mo**: Claude Pro + all free tiers	Everything in this guide. Ship personal projects and portfolio pieces.
Freelancer / launch	**About $65/mo**: Claude Pro $20 + Vercel Pro $20 + Supabase Pro $25	Client sites, paid products, apps with real users.
Power user	$100 to $200/mo Claude + Vercel Pro + Supabase Pro	Building all day, running parallel Claude Code sessions.

Gotchas that cost beginners money or time

1) Vercel Hobby is for non-commercial use: if you charge customers, use Pro. 2) Supabase free projects pause after a week idle, so a quiet demo site can go down. 3) Annual billing saves money, but start monthly until you know you'll keep using it. 4) Check pricing pages before buying: plans and limits change.

CHAPTER 03

Set up your machine

About 30 minutes, once. Do these in order and you'll never think about setup again.

1. **Create a Claude account and subscribe**
   Go to claude.ai, sign up, and pick Pro. Download the Claude desktop app (macOS or Windows) from claude.com/download.
2. **Create a GitHub account**
   Sign up at github.com. Pick a username you'd be happy to put on a resume or in your Twitter bio.
3. **Install Git**
   macOS: open Terminal and run `git --version`, and accept the prompt to install developer tools. Windows: install Git for Windows from git-scm.com.
4. **Install Node.js (LTS)**
   Download the LTS version from nodejs.org. Most web projects need it. Check with `node --version`.
5. **Install Claude Code**
   Easiest: open the Claude desktop app and click the **Code** tab (Chapter 4). To also use the terminal version, run the install command below.
6. **Create Vercel and Supabase accounts**
   Sign up at vercel.com and supabase.com using **Continue with GitHub** so everything is linked.
7. **Install Obsidian**
   Download from obsidian.md. Create a vault (a folder) called *Projects* anywhere you can find it.

```
# Terminal install of Claude Code
# macOS / Linux / WSL:
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell:
irm https://claude.ai/install.ps1 | iex

# Then open a NEW terminal window and check it worked:
claude --version
```

10 terminal commands that cover 90% of what you'll do

Command	What it does
`pwd`	Shows the folder you're in
`ls` (Windows CMD: `dir`)	Lists files in the current folder
`cd folder-name`	Moves into a folder (`cd ..` goes back up)
`mkdir my-app`	Creates a new folder
`claude`	Starts Claude Code in the current folder
`npm install`	Installs a project's packages
`npm run dev`	Starts your app locally, usually at http://localhost:3000
`git status`	Shows what changed since your last save
`Ctrl + C`	Stops whatever is running
`clear`	Cleans up the screen

Stuck on something terminal-y?

Ask Claude. "I'm on [Mac/Windows]. I ran this command and got this error: [paste]. What does it mean and what do I type next?" It's a patient tutor for exactly this.

PART 1: FOUNDATIONS

CHAPTER 04

Claude Code: terminal vs desktop app

Same engine, two different ways to drive it. Here's how to choose.

First, a common point of confusion: **Claude chat** (claude.ai) is where you talk and build Artifacts in a browser tab. **Claude Code** works directly inside a folder on your computer: it reads your project files, edits them, runs commands, and can commit to git. Claude Code is available in the terminal, in the desktop app's Code tab, in VS Code or JetBrains, and on the web. All of them use the same underlying Claude Code, so your CLAUDE.md files, settings and connectors work everywhere.

	Terminal (CLI)	Desktop app (Code tab)
What it is	Text-based Claude Code you run by typing `claude` in a terminal	A graphical app with a chat panel, file diffs, a built-in browser preview and terminal pane
Setup	One install command (see Chapter 3)	Download the app, sign in, click Code. No Node.js or CLI install needed
Reviewing changes	Read diffs as text in the terminal	Visual diff view: click the +/- line counts, review file by file, comment on specific lines

	Terminal (CLI)	Desktop app (Code tab)
Seeing your app	You open the browser yourself	Dev server opens in a Browser pane next to the chat; Claude can look at the running app and iterate
Permission control	Modes include a CLI-only `dontAsk`; `Shift+Tab` cycles modes	Mode picker: Manual, Accept edits, Plan, Auto, and Bypass
Parallel work	Open several terminals; scripting and piping are strengths	Multiple sessions side by side, each optionally in its own git worktree
Extras	Scriptable: pipe logs in, run in CI, automate	Scheduled tasks, side chats, PR monitoring, cloud sessions, phone dispatch
Best for	Power users, automation, remote servers	**Beginners** and anyone who likes seeing what changed

Which should you start with?

Start with the desktop app's Code tab. You can see every change, approve edits, and watch your app update live, which builds intuition fast. Add the terminal version once you're comfortable (you'll want it for scripting and quick one-liners). You can even run both on the same project, and a terminal session can hand off to the desktop app with `/desktop`.

Your first session in the desktop app

1. **Open the Code tab**
   Launch the Claude app, sign in, click **Code** in the sidebar.
2. **Pick Local and a folder**
   Choose the **Local** environment, then Open folder and select (or create) your project folder.
3. **Pick a model and a permission mode**
   Start in **Manual** so you approve each change, or **Accept edits** once you trust the flow. Use **Plan** mode before big changes.
4. **Ask for something small**
   "Create a simple homepage for a coffee shop and run it so I can see it." Review the diff, approve, and look at the preview.

Where else Claude Code runs

- **VS Code / Cursor / JetBrains:** extensions with inline diffs, if you already live in an editor.
- **Web (claude.ai/code) and the Claude mobile app:** run long tasks in the cloud against a GitHub repo, no local setup, and check back later.
- **Claude in Chrome:** lets Claude work in your real browser, useful for testing a live site or filling in dashboards.

GitHub: your safety net

GitHub stores your code and every version of it. Think of it as a save game with unlimited undo.

Five words you need

Word	Meaning
Repository (repo)	A project folder tracked by git, plus its history, stored on GitHub
Commit	A saved snapshot with a message, like "add login page"
Branch	A parallel copy where you can experiment without touching the working version
Push / Pull	Push uploads your commits to GitHub. Pull downloads new ones
Pull request (PR)	A proposal to merge a branch into the main one, with a place to review the changes

Connect your project to GitHub

1. **Create the repo**
   On github.com click **New**. Name it, choose Private (you can go public later), and don't add a README yet.
2. **Let Claude do the git commands**
   In Claude Code say: *"Initialize git here, add a good .gitignore for this stack, make the first commit, and connect it to my GitHub repo [paste URL]. Walk me through any login step."*
3. **Log in once**
   Git will ask you to authenticate. The simplest way is installing the GitHub CLI (`gh`) and running `gh auth login`.
4. **Push**
   Ask: *"Push this to GitHub."* Refresh your repo page and your code is there.

The daily workflow

```
1. New feature?  ->  "Create a branch called feature/login"
2. Build it with Claude, run it, make sure it works
3. "Commit this with a clear message"
4. "Push and open a pull request"
5. Look at the PR on GitHub, then merge
6. Vercel auto-deploys the merged main branch (Chapter 6)
```

Undo buttons (memorize these ideas, not the commands)

Situation	Say this to Claude
You hate the last change	"Undo my last uncommitted change in [file]."
You committed something broken	"Revert the last commit safely and explain what you did."
App worked yesterday, broken now	"Compare to the commit from yesterday and find what broke it."
You want to try something risky	"Create a new branch so I can experiment without risking main."

Never commit secrets

API keys, database passwords and `.env` files must never go into git. Make sure `.env*` is in your `.gitignore` before your first commit. If a key ever leaks (even from a private repo), rotate it immediately. Anything pushed to a public repo should be treated as public forever.

PART 2: THE TOOLS

CHAPTER 06

Vercel: put it on the internet

Connect your GitHub repo once, and every push becomes a live website.

Deploy in 5 steps

1. **Push your project to GitHub**
   Vercel deploys from a repo (Chapter 5).
2. **Import it**
   At vercel.com click **Add New**, then **Project**, and pick your repo. Vercel detects the framework (Next.js, Vite, plain HTML) automatically.
3. **Add environment variables**
   Before deploying, open **Environment Variables** and add any keys your app needs (for example your Supabase URL and public key).
4. **Click Deploy**
   In about a minute you get a live `yourproject.vercel.app` link.
5. **Push again to update**
   Every push to **main** redeploys your production site automatically.

Why Vercel feels like magic

- **Preview deployments:** every branch and pull request gets its own temporary URL, so you can test changes before they go live, or send a link to a friend for feedback.
- **Instant rollbacks:** if a deploy goes wrong, promote a previous deployment back to production in a couple of clicks.
- **Custom domains:** add yours in Project Settings, then Domains. Vercel tells you exactly which DNS records to set at your registrar.
- **Logs:** build errors and runtime errors are visible in the dashboard. Copy them straight into Claude.

Which framework should you pick?

You don't have to decide: tell Claude what you're making and let it choose, but know the basics.

If you're building...	Ask Claude for...
A single landing page or portfolio	Plain HTML, CSS and JS (simplest to understand and deploy)
A site with several pages, a blog, or a SaaS	Next.js with the App Router and Tailwind CSS
A fast interactive tool or dashboard	Vite + React

PROMPT: MAKE IT DEPLOY-READY

Prepare this project for deployment on Vercel. Make sure the build command works locally, list every environment variable it needs in a .env.example file (without real values), and tell me exactly what to enter in the Vercel dashboard.

When a deploy fails

- Open the failed deployment in Vercel and copy the **build log** (the red error at the bottom).
- Paste it into Claude: *"This Vercel build failed. Here's the log. Find the cause and fix it, then verify with a local build."*
- The most common causes: a missing environment variable, a typo in an import, or a package that wasn't installed.

Heads up on plans

Vercel's free Hobby plan is for personal, non-commercial projects. If your site earns money or you build it for a paying client, use Pro ($20/mo). Also note the platform has usage limits, so a viral post can hit them.

Supabase: add a database and logins

When your app needs to remember things or know who a user is, Supabase gives you a real database and authentication without running a server.

What you get

Feature	What it does	Example
Database (Postgres)	Stores your data in tables	Saved links, orders, habits, waitlist emails
Auth	Sign-up, login, password reset, social login	"Sign in with Google"
Storage	Holds files like images and PDFs	Profile photos, uploads
Edge Functions	Small server-side functions	Calling the Claude API without exposing your key

Set it up

1. **Create a project**
   At supabase.com click **New project**, name it, set a strong database password (save it in a password manager), and pick a region near your users.
2. **Find your keys**
   In **Project Settings, API** you'll see the project URL and a public (anon / publishable) key. Those two are safe to use in your frontend.
3. **Store keys in .env**
   Ask Claude: *"Add Supabase to this project using environment variables. Create .env.local and .env.example, and make sure they're git-ignored."*
4. **Create tables with a prompt**
   Describe the data in plain English and let Claude write the SQL. Run it in the Supabase SQL editor, or have Claude apply it as a migration.
5. **Turn on Row Level Security (RLS)**
   This is the most important step. RLS decides which rows each user can see and change.

PROMPT: TABLES + SECURITY TOGETHER

Create a Supabase table called bookmarks with id, user_id (references auth.users), url, title, tags, and created_at. Enable row level security so each signed-in user can only select, insert, update and delete their own rows. Give me the SQL and explain each policy in one sentence.

The key rule that prevents disasters

Supabase gives you a **public (anon/publishable) key** and a **service role (secret) key**. The public key can live in your frontend because RLS protects the data. The service role key bypasses all security: never put it in frontend code, never commit it, never paste it into screenshots or posts. Without RLS enabled, anyone with your public key can read your tables.

Add login to your app

PROMPT

Add Supabase email + password sign-up and login to this app, plus a logout button. Redirect logged-out users away from the dashboard page. Show helpful error messages. Keep the design consistent with the rest of the app.

Free tier reality check

The Free plan gives you 2 active projects and a 500 MB database, which is plenty for learning and small apps. Free projects **pause after one week of inactivity** (you can restore them from the dashboard). Before showing an app to real users, move to Pro (from $25/mo) so it never sleeps.

PART 2: THE TOOLS

CHAPTER 08

Obsidian: your project brain

Obsidian is a free notes app that stores everything as plain Markdown files in a normal folder. That's exactly why it pairs so well with Claude.

Most people use notes apps as a graveyard. For builders, notes become **fuel**: a clear spec makes Claude's output dramatically better, and a saved prompt library means you never start from scratch. Because an Obsidian "vault" is just a folder of .md files, Claude Code can read and write it directly. Obsidian is optional, but it's the easiest way to stay organized as projects multiply.

A simple vault structure

```
Projects/                    <- your vault
  00 Inbox/                  quick ideas, dump anything here
  Projects/
    Bookmark Vault/
      Spec.md                what we're building and why
      Decisions.md           "we chose Supabase because..."
      Prompts.md             prompts that worked
      Bugs.md                problems and the fixes
  Prompt Library/            reusable prompts by category
  Design Notes/              palettes, fonts, references
  Daily/                     a tiny dev log per day
```

Four habits that pay off

- **Write the spec in Obsidian first.** Ask Claude to interview you and produce a Spec.md, then tell Claude Code: *"Read Spec.md and build v0 only."*
- **Keep a prompt library.** When a prompt gives you a great result, paste it into *Prompt Library* with a note on what it's good for.
- **Log decisions and bugs.** Future you (and future Claude sessions) will thank you. One line per entry is enough.
- **Use links.** Type `[[Spec]]` to link notes. The graph view makes your project's structure visible.

> **PROMPT: TURN AN IDEA INTO A SPEC FILE**
> I want to build [idea]. Interview me with up to 6 questions, then write a Spec.md in markdown with: goal, target user, must-have features, nice-to-haves, screens, data model, visual style, and a build order of small steps. Save it to my Obsidian vault folder at [path].

Connecting Claude Code to your vault

Point Claude Code at the vault folder (or a project folder inside it) when you start a session, and it can read your specs and update your notes like any other files. Keep your code project folder separate from your vault, and just tell Claude where each one lives in your CLAUDE.md.

PART 2: THE TOOLS

CHAPTER 09

Claude skills

A skill is a folder of instructions, and sometimes scripts, that teaches Claude to do one kind of job really well, every time.

Every skill has a `SKILL.md` file: a name, a one-line description of when to use it, and the instructions. Claude reads the short descriptions of your enabled skills and loads the full instructions only when a task matches. Think of skills as plug-in expertise: "design like a pro," "make a clean PDF," "build me a skill."

Where to find skills

Source	What's there
Claude app: Customize, Skills	Built-in and example skills you can toggle on. Available on Free, Pro, Max, Ultra, Team and Enterprise (needs code execution / file creation turned on)
github.com/anthropics/skills	Anthropic's public skills repo: examples for design, document creation (docx, pdf, pptx, xlsx), testing web apps, MCP servers and more, plus a template and the spec
Claude Code plugin marketplaces	In Claude Code run `/plugin marketplace add anthropics/skills` and then install bundles from it
Community repos on GitHub	Search "claude skills" or "agent skills". Quality varies, so read before installing
Your own	Write a SKILL.md for any workflow you repeat. The skill-creator skill (included below) walks you through it

How to install a skill

In the Claude app (chat and Claude Code sync):

1. **Turn on the prerequisites**
   Settings, Capabilities, switch on **Code execution and file creation** (newer layouts call it *Create and edit files*).
2. **Open Skills**
   Go to **Customize, Skills**.
3. **Upload**
   Click **+**, then **Create skill**, then **Upload a skill** and choose the .zip file. The folder inside the zip must match the skill name and contain SKILL.md.
4. **Toggle it on**
   It appears in your list. Switch it on, then mention it in a prompt: "Use the hallmark skill to design a landing page for..."

Skills you enable in your Claude account also load in Claude Code when you're signed in with the same account (run `/skills` to see them). You can also keep skills as folders in your user or project `.claude/skills/` directory; see the Claude Code skills docs for current details.

> **Security note**
> Skills can contain scripts that Claude runs on your machine. Only install skills from sources you trust, and skim the SKILL.md (and any scripts) before enabling one. The skills in this guide are open-source licensed so you can read every file.

Two free skills included with this guide

Skill	What it does	Use it when	License
hallmark	Design skill that pushes Claude past generic, templated-looking AI websites: a hard floor against default layouts, plus a dealt design direction so each build looks different.	Building a new website or landing page that should look designed, not generated. Try: *"Use hallmark to design a landing page for..."*	MIT
skill-creator	Helps you create new skills from scratch, improve existing ones, test them with evals, and tune descriptions so they trigger correctly.	You've got a repeatable workflow (your brand style, your deploy checklist) and want Claude to do it your way every time.	Apache 2.0

Both come as ready-to-upload .zip files alongside this guide (hallmark.zip and skill-creator.zip). Upload them using the steps above. Each includes its original license file.

Make your own in 5 minutes

> **PROMPT (WITH SKILL-CREATOR ENABLED)**
> Use skill-creator to help me make a skill called my-brand-style. It should make every page I build use my palette (#0F0E17, #D97757, #7C6CF0), Inter for text, rounded corners, and a dark theme. Interview me for any details you need, then package it as a zip I can upload.

PART 3: BUILD AND SHIP

CHAPTER 10

The build loop and prompting that works

Every good vibe-coded project follows the same five beats, and good prompts are specific about the outcome and relaxed about the method.

1	**Idea** One sentence: who is it for and what does it do?
2	**Spec** Ask Claude to turn it into a short plan (or write Spec.md in Obsidian). Edit it before any code exists.
3	**Build v0** Ask for the smallest version that works: one screen, one core feature. Ugly is fine.
4	**Run and react** Open it. Click everything. Note what's wrong, missing, or delightful. Screenshots help a lot.
5	**Refine** Change one thing at a time. Describe the change, run again, repeat. Commit after every win.

The 5-part prompt

Part	What to include	Example
Goal	What you're making and who it's for	A landing page for my dog-walking business
Vibe	Look, feel, references, mood	Warm, playful, rounded shapes, lots of whitespace
Must-haves	Features and content that have to exist	Pricing cards, booking form, 3 testimonials
Constraints	Tech, size, things to avoid	Single HTML file, no frameworks, mobile first
Done when	How you'll know it's right	Looks good on a phone and the form validates email

WEAK PROMPT

Make me a nice website for my business.

STRONG PROMPT

Build a one-page website for my dog-walking business, Happy Paws. Vibe: warm, playful, rounded corners, soft yellow and navy, big friendly headline. Include: hero with a booking button, 3 pricing cards, 3 testimonials, and an email signup. Single HTML file, no frameworks, mobile first. Start with the hero and pricing only.

Phrases that level up your results

- **"Ask me questions first."** Fills gaps in your idea before building.
- **"Plan first, don't write code yet."** Read the plan in 30 seconds and avoid a 30-minute wrong turn. In Claude Code, use Plan mode.
- **"Give me 3 different design directions."** Explore the look before committing.
- **"Keep everything else exactly the same."** Protects what already works while you change one thing.
- **"Explain what you changed in two sentences."** You learn the code as you go.
- **Show, don't tell.** Paste a screenshot of a site you like and say "make it like this, but with my colors."

CHAPTER 11

Capstone: build and ship a real app

Let's put every tool together: a private bookmark vault with logins, built in Claude Code, stored on GitHub, hosted on Vercel, backed by Supabase, planned in Obsidian.

What you'll have at the end: a live website where you sign up, save links with tags, search them, and only you can see your data. About 2 to 3 hours the first time.

Step 1: Plan it in Obsidian

> **PROMPT 1 (CLAUDE CHAT OR CODE)**
> I want to build a private bookmark vault: sign up and log in, save a link with title and tags, search and filter by tag, delete bookmarks. Clean, dark, minimal design. Interview me with up to 5 questions, then write Spec.md with features, screens, data model and a build order. Save it in my Obsidian vault.

Step 2: Start the project and repo

> **PROMPT 2 (CLAUDE CODE, IN A NEW EMPTY FOLDER)**
> Read my Spec.md at [path]. Set up a new Next.js project with Tailwind CSS in this folder, create a CLAUDE.md with the stack and rules, initialize git with a proper .gitignore, and connect it to my GitHub repo [URL]. Build nothing else yet. Show me how to run it locally.

Step 3: Build v0 with fake data

> **PROMPT 3**
> Build the main bookmarks page with hardcoded sample data: an add-bookmark form, a list of bookmark cards (title, domain, tags), a search box, and tag filter chips. Dark, minimal, mobile friendly. Commit when it works.

Step 4: Connect Supabase

> **PROMPT 4**
> Add Supabase using environment variables (create .env.local and .env.example, both git-ignored). Create a bookmarks table with row level security so users only access their own rows. Give me the SQL to run, and replace the hardcoded data with real database reads and writes.

Step 5: Add login

PROMPT 5

Add email + password sign-up and login with Supabase Auth, a logout button, and protect the bookmarks page so logged-out users are redirected to a login page. Handle errors gracefully.

Step 6: Polish

PROMPT 6

Polish pass: add loading and empty states, confirm-before-delete, keyboard shortcut to focus search, and a favicon. Check every page at 375px width. Then review the whole app for security issues and tell me what you find.

Step 7: Ship it

1. **Merge and push**
   Ask Claude: *"Commit everything, push, and open a pull request. Then tell me what to check."* Merge the PR on GitHub.
2. **Import into Vercel**
   Add New Project, pick the repo, and paste your two Supabase env vars. Deploy.
3. **Allow your live URL in Supabase**
   In Supabase, Authentication, URL Configuration, add your Vercel URL as the site URL so login redirects work.
4. **Test as a stranger**
   Open the live URL in a private window. Sign up with a new email, add a bookmark, log out, log in. Try it on your phone.
5. **Log it**
   Add what you learned and anything that broke to Decisions.md and Bugs.md in Obsidian.

You just did it

That is a full-stack app with authentication, a database, version control, continuous deployment and a documented spec, built by describing what you wanted. Every project from here is a variation.

CHAPTER 12

Fix anything: the debugging playbook

Things will break. With the right information, it's usually a 60-second fix.

The 4-step fix

- **Copy the error exactly.** The red text in the browser console, terminal, or Vercel log is the most useful thing you can paste.
- **Say what you expected and what happened.** "I clicked Save and nothing appeared in the list."
- **Add a screenshot** if it's visual. Layout bugs are easier to see than to describe.
- **Ask for the cause, then the fix.** "Explain why this happened, then fix it without changing anything else."

> DEBUG PROMPT TEMPLATE
>
> Expected: [X]. Actual: [Y]. Error / screenshot: [paste]. Explain the likely cause in plain English, fix it, and verify the fix by running the app or tests. Don't change anything unrelated.

Common problems

Problem	Try this
Claude changed things you liked	"Revert that last change." Next time add "keep everything else exactly the same."
Same bug keeps coming back	Ask for the root cause, then start a fresh chat with only the relevant file and the bug description.
Works locally, fails on Vercel	Usually a missing environment variable or a build error. Paste the Vercel build log to Claude.
Data won't save or load (Supabase)	Almost always row level security policies or a wrong key. Ask: "Check my RLS policies against what the app is trying to do."
Login redirects to the wrong place	Add your live URL to Supabase's URL Configuration (site URL and redirect URLs).
Design looks generic	Give references and a mood word, ask for 3 directions, or use the hallmark skill.
Code got huge and messy	"Refactor into clear files without changing behavior, then verify everything still works."
Claude is confused or repeating itself	Long chats drift. Start a new session, paste a short project summary, continue.

Ask Claude to test its own work

Add: "After the change, run the app (or tests) and tell me how you verified it." In Claude Code it can actually start the app, read errors, and fix them itself.

PART 3: BUILD AND SHIP

CHAPTER 13

15 cool things to build

Each is a good weekend project and a great demo to post. Steal the prompt starters.

#	Idea	Prompt starter
1	**Mood-reactive landing page**	*Build a landing page whose gradient and headline shift as the user scrolls, like a sunrise to a night sky.*
2	**Interactive 3D product viewer**	*Build a draggable 3D rotating sneaker viewer using Three.js with hotspot labels and a color picker.*
3	**AI roast / compliment generator**	*Build a web app where users enter their job and get a funny, kind roast. Use the Claude API via a serverless function.*
4	**Personal dashboard**	*Build a beautiful start page with a clock, weather, to-do list and daily quote. Glassmorphism style.*
5	**Mini browser game**	*Build a one-button endless runner in a single HTML file with neon visuals and a saved high score.*
6	**Generative art playground**	*Build a canvas art tool with sliders for color, noise and symmetry, plus a download-PNG button.*
7	**Habit tracker with streaks**	*Build a habit tracker with a GitHub-style heatmap and a confetti burst when I hit a new streak. Store it in Supabase.*
8	**Link-in-bio page**	*Build a link-in-bio page with animated cards, a profile photo, and a dark/light toggle.*
9	**Text-to-flashcards tool**	*Build an app where I paste notes and Claude turns them into flip-style flashcards and a short quiz.*
10	**Music visualizer**	*Build an audio visualizer with the Web Audio API that reacts to a song I upload. Three visual modes.*
11	**Expense splitter**	*Build a mobile-first bill splitter with tip, uneven splits, and a shareable summary.*
12	**Portfolio with personality**	*Build my portfolio with a custom cursor, smooth page transitions, and project cards that react on hover.*
13	**Chrome extension**	*Build a Chrome extension that hides distracting feeds on sites I choose and shows a motivational quote instead.*

#	Idea	Prompt starter
14	**Waitlist page with a database**	*Build a waitlist page that saves emails to Supabase with validation and a thank-you animation.*
15	**Mini SaaS tool**	*Build a tool that turns a rough product description into 5 landing-page headlines. Add login and a free usage limit.*

PART 3: BUILD AND SHIP

CHAPTER 14

Pro habits and the pre-launch checklist

Small routines separate people who ship from people who get stuck.

- **Small steps, always.** One feature per prompt. If a prompt has "and" five times, split it.
- **Commit after every win.** Your safety net for fearless experimenting.
- **Keep a CLAUDE.md.** Claude Code reads it at the start of every session. Put your stack, style rules and "don't do this" notes there.
- **Read what it writes.** Skim it and ask "what does this part do?" You'll learn faster than you expect.
- **Fresh session for fresh problems.** Long conversations get cluttered.
- **Save prompts that worked.** Your prompt library is a compounding asset.

A starter CLAUDE.md

```
# Project: My Cool App

## Stack
- Next.js (App Router), Tailwind CSS, Supabase
- Deploys to Vercel from GitHub main

## Style
- Dark theme, soft gradients, rounded corners, mobile-first

## Rules
- Small changes; don't rewrite files unprompted
- Explain changes in 2 sentences
- Never hardcode secrets; use .env.local and keep .env.example updated
- Commit working versions with clear messages
- Specs live in my Obsidian vault at [path]
```

Pre-launch checklist

Check	Why
No secrets in code or git history	Search the repo for keys. Rotate anything that ever leaked.
Row level security on every Supabase table	Otherwise your public key can read your data.
Service role key only on the server	It bypasses all security.
Tested on a real phone and in a private window	Catches layout and login bugs you'd never see otherwise.
Ask Claude: "Review this for security issues"	A free second pair of eyes before strangers use it.
Right plan for the use case	Vercel Pro if commercial, Supabase Pro if users depend on it.
Privacy basics	If you collect emails or accounts, add a short privacy note and don't store more than you need.

Four-week level-up path

Week	Focus	Goal
1	Claude chat + Artifacts, GitHub account, first repo	Ship 3 tiny things
2	Claude Code, git, Vercel	One project live on its own URL
3	Supabase tables and logins, Obsidian specs	An app that remembers its users
4+	Skills, Claude API inside your apps, custom domain	Something other people actually use

APPENDIX

CHAPTER 15

Cheat sheet and glossary

Screenshot this and keep it nearby.

THE LOOP

Idea, Spec, Build v0, Run and react, Refine. One change per prompt.

THE 5-PART PROMPT

Goal, Vibe, Must-haves, Constraints, Done-when.

MAGIC PHRASES

"Ask me questions first."
"Plan before you code."
"Give me 3 directions."
"Keep everything else the same."
"Explain in 2 sentences."

WHEN IT BREAKS

Paste the exact error. Expected vs actual. Add a screenshot. Cause first, then fix.

SHIP

Commit, push to GitHub, Vercel auto-deploys. Secrets live in .env only.

START WITH

Claude Pro ($20) + free GitHub, Vercel Hobby, Supabase Free, Obsidian. Desktop app Code tab first, terminal second.

Glossary

Term	Meaning
API	A way for programs to talk to each other. The Claude API lets your app call Claude.
Auth	Sign-up and login: proving who a user is.
Build	Turning your source code into the optimized files a website runs.
CLAUDE.md	A file Claude Code reads at the start of every session with your project's rules.
Deploy	Publishing your app so anyone can visit it.
Environment variable	A secret or setting kept outside your code, like an API key, stored in .env.
Frontend / backend	What users see and click / the server and database behind it.
MCP / connector	A standard that lets Claude plug into other tools, like GitHub, Vercel or Supabase.
Migration	A saved change to your database structure, so it can be repeated or undone.
RLS	Row Level Security: database rules saying which user can see or edit which row.
SQL	The language used to talk to a database.
Terminal	The text window where you type commands.



Now go build something.

Post it. Share the prompt. Show your process.

Claude and Claude Code are products of Anthropic. Prices and features change; always check each company's official site.
Vercel, Supabase, GitHub and Obsidian are trademarks of their respective owners. This guide is independent and not affiliated with them.